

Strukturni paterni u StudyBuddy sistemu

1. Uvod

StudyBuddy sistem omogućava korisnicima kreiranje sesija učenja, prijavljivanje na sesije, evidentiranje prisustva, pregled obavještenja i upravljanje osnovnim podacima sistema. U prethodnim projektnim zadacima kreiran je osnovni klasni dijagram koji prikazuje glavne klase sistema, kao što su **Korisnik**, **SesijaUcenja**, **PrijavaNaSesiju**, **Prisustvo**, **Predmet**, **Lokacija**, **Uloga** i **Obavjestenje**.

Kako sistem postaje složeniji, javlja se potreba za boljom organizacijom odnosa između klasa. Strukturni paterni omogućavaju da se klase povežu na pregledniji način, da se smanji međusobna zavisnost i da se sistem lakše proširuje u budućnosti. U StudyBuddy sistemu odabrana su dva strukturalna paterna: **Facade** i **Adapter**.

2. Potreba za dodavanjem strukturalnih paterna

U postojećem sistemu određene funkcionalnosti uključuju više povezanih klasa. Na primjer, funkcionalnost prijavljivanja korisnika na sesiju ne koristi samo klasu **PrijavaNaSesiju**, nego uključuje i klasu **Korisnik**, klasu **SesijaUcenja**, provjeru broja slobodnih mjesta i kreiranje obavještenja. Ukoliko bi svaki dio sistema direktno komunicirao sa svim ovim klasama, dijagram bi postao složeniji, a sistem teži za održavanje.

Zbog toga se uvodi **Facade patern**, koji omogućava jednostavniji pristup složenom podsistemu sesija učenja. Također, sistem ima klasu **Obavjestenje**, ali u praksi slanje obavještenja može zavisiti od eksternog email servisa. Kako se postojeće klase ne bi morale prilagođavati vanjskom servisu, uvodi se **Adapter patern**.

Na ovaj način se postojeći sistem ne mijenja u osnovnoj strukturi, nego se nadograđuje dodatnim klasama koje poboljšavaju njegovu fleksibilnost i preglednost.

3. Moguća primjena strukturalnih paterna u StudyBuddy sistemu

Strukturalni paterni omogućavaju bolju organizaciju odnosa između klasa i objekata. Njihova primjena u StudyBuddy sistemu može pomoći u smanjenju zavisnosti između klasa, lakšem povezivanju sa vanjskim servisima, jednostavnijem proširenju funkcionalnosti i boljoj organizaciji složenih dijelova sistema.

Adapter patern

Adapter patern bi se u StudyBuddy sistemu mogao koristiti za povezivanje sistema sa eksternim servisima za slanje obavještenja. Postojeća klasa **Obavjestenje** predstavlja podatke o obavještenju, dok vanjski email servis može imati drugačiji interfejs za slanje poruka. Uvođenjem klase

EmailSenderAdapter omogućava se da StudyBuddy sistem koristi jedinstveni interfejs IObavještenjeSender, dok adapter prilagođava podatke formatu koji očekuje eksterni servis.

Ovaj patern je posebno koristan jer omogućava kasnije dodavanje drugih načina slanja obavještenja, kao što su SMS ili push notifikacije, bez promjene postojećih model klasa.

Bridge patern

Bridge patern bi se mogao primijeniti u slučaju da se funkcionalnosti sistema trebaju razdvojiti od načina njihove tehničke realizacije. Na primjer, sistem može imati različite vrste obavještenja, kao što su podsjetnik, prijava, promjena termina ili otkazivanje, dok način slanja može biti email, SMS ili push notifikacija.

U tom slučaju bi se mogla odvojiti apstrakcija obavještenja od konkretne implementacije slanja. Time bi se omogućilo da se nove vrste obavještenja i novi načini slanja dodaju nezavisno jedni od drugih. Ipak, za trenutni obim StudyBuddy sistema ovaj patern nije izabran za dijagram jer bi dodatno zakomplikovao model.

Composite patern

Composite patern se koristi kada objekti mogu biti organizovani u hijerarhijsku strukturu. U StudyBuddy sistemu ovaj patern bi se mogao koristiti ako bi se predmeti organizovali po oblastima, modulima ili grupama. Na primjer, jedan veći predmet ili oblast mogao bi sadržavati više manjih tema ili podpredmeta.

Tako bi se mogla formirati struktura u kojoj se pojedinačni predmeti i grupe predmeta tretiraju na sličan način. Međutim, u postojećem modelu klasa Predmet nema hijerarhijsku strukturu, pa Composite nije najprirodniji izbor za trenutnu verziju sistema.

Decorator patern

Decorator patern bi se mogao koristiti za dinamičko dodavanje dodatnih mogućnosti objektima bez mijenjanja njihove osnovne klase. U StudyBuddy sistemu to bi se moglo primijeniti na klasu SesijaUcenja.

Na primjer, osnovna sesija učenja može imati dodatne karakteristike: online materijale, podsjetnik, dodatni opis, oznaku prioriteta ili poseban način organizacije. Umjesto da se osnovna klasa SesijaUcenja proširuje velikim brojem atributa, Decorator bi omogućio dodavanje tih mogućnosti kroz posebne klase. Ipak, ovaj patern nije izabran za finalni dijagram jer trenutni sistem nema dovoljno različitih dodataka sesijama da bi njegova primjena bila opravdana.

Facade patern

Facade patern je veoma prikladan za StudyBuddy sistem jer funkcionalnosti vezane za sesije učenja uključuju više različitih klasa. Na primjer, prijavljivanje korisnika na sesiju uključuje klase

Korisnik, SesijaUcenja, PrijavaNaSesiju i Obavjestenje. Također može uključivati provjeru slobodnih mjesta i slanje obavještenja.

Zbog toga se uvodi klasa **SesijaFacade**, koja omogućava jednostavniji pristup ovim funkcionalnostima. Umjesto da drugi dijelovi sistema direktno komuniciraju sa više klasa, oni mogu koristiti fasadu koja koordinira čitav proces.

Proxy patern

Proxy patern bi se mogao koristiti za kontrolu pristupa određenim funkcionalnostima sistema. U StudyBuddy sistemu neke funkcionalnosti ne bi trebale biti dostupne svim korisnicima. Na primjer, samo moderator ili administrator može kreirati određene sesije, uređivati podatke ili upravljati korisnicima.

Proxy klasa bi mogla provjeravati ulogu korisnika prije nego što dozvoli pristup određenoj funkcionalnosti. Na taj način bi se u sistem mogao uvesti dodatni sloj kontrole pristupa. Ipak, pošto sistem već ima klasu Uloga i enum TipUloge, ovaj patern nije izabran za finalni dijagram kako se ne bi nepotrebno ponavljala postojeća logika autorizacije.

4. Odabrani strukturalni paterni

Facade patern se koristi kada je potrebno pojednostaviti pristup složenom podsistemu. Umjesto da klijent direktno koristi više različitih klasa, uvodi se jedna fasadna klasa koja objedinjuje potrebne operacije.

U StudyBuddy sistemu ovaj patern je primijenjen kroz klasu **SesijaFacade**. Ova klasa predstavlja jednostavan ulaz u funkcionalnosti vezane za sesije učenja. Ona koristi postojeće klase sistema, ali sakriva složenost njihove međusobne komunikacije.

Klasa **SesijaFacade** koristi sljedeće klase:

- **Korisnik**
- **SesijaUcenja**
- **PrijavaNaSesiju**
- **Predmet**
- **Lokacija**
- **Obavjestenje**
- **IObavjestenjeSender**

U dijagramu su u klasi **SesijaFacade** definisane metode:

- Metoda **kreirajSesiju** omogućava kreiranje nove sesije učenja na osnovu korisnika koji je kreira, odabranog predmeta i lokacije

- Metoda **prijaviKorisnikaNaSesiju** objedinjuje proces prijave korisnika na sesiju
- Metoda **provjeriSlobodnaMjesta** provjerava da li je moguće dodati novog učesnika.
- Metoda **otkaziSesiju** omogućava otkazivanje postojeće sesije.

Prednost ovog paterna je u tome što se složen proces rada sa sesijama svodi na korištenje jedne klase. Time se smanjuje direktna zavisnost drugih dijelova sistema od velikog broja model klasa.

Adapter patern se koristi kada postojeći sistem treba koristiti klasu ili servis sa drugačijim interfejsom. Adapter služi kao posrednik između sistema i klase koja se ne može direktno koristiti bez prilagođavanja.

U StudyBuddy sistemu ovaj patern je primijenjen na dio sistema koji se odnosi na obavještenja. Postojeća klasa **Obavjestenje** čuva podatke o obavještenju, kao što su naslov, sadržaj, datum slanja, korisnik i tip obavještenja. Međutim, stvarno slanje obavještenja može se izvršavati preko eksternog email servisa.

Problem je u tome što eksterni email servis može imati drugačiju metodu za slanje poruke. Na primjer, u dijagramu klasa **EksterniEmailServis** posjeduje metodu:

- +sendEmail(primalac : string, naslov : string, sadrzaj : string) : void

S druge strane, StudyBuddy sistem radi sa objektom klase **Obavjestenje**. Zbog toga je uveden interfejs **IObavjestenjeSender** koji definiše metodu:

- +posalji(obavjestenje : Obavjestenje) : void

Klasa **EmailSenderAdapter** implementira interfejs **IObavjestenjeSender** i prilagođava objekat klase **Obavjestenje** formatu koji očekuje eksterni email servis. Ona sadrži atribut:

- -emailServis : EksterniEmailServis

i metodu:

- +posalji(obavjestenje : Obavjestenje) : void

Na taj način ostatak sistema ne mora znati detalje rada eksternog email servisa. Sistem koristi interfejs **IObavjestenjeSender**, a adapter preuzima odgovornost za prilagođavanje podataka.

Ova struktura omogućava da se u budućnosti doda novi način slanja obavještenja, na primjer SMS ili push notifikacije, bez izmjene postojećih model klasa.

5. Veza između odabranih paterna

U kreiranom dijagramu oba paterna su međusobno povezana. Klasa **SesijaFacade** koristi interfejs **IObavjestenjeSender** kako bi mogla poslati obavještenje nakon kreiranja sesije, prijave korisnika ili otkazivanja sesije.

Time se postiže bolja organizacija sistema:

- **Facade** pojednostavljuje rad sa procesima vezanim za sesije.
- **Adapter** omogućava povezivanje sistema sa eksternim servisom za slanje email obavještenja.
- **SesijaFacade** ne zavisi direktno od klase **EksterniEmailServis**, nego od interfejsa **IObavjestenjeSender**.

Ovakvo rješenje povećava fleksibilnost sistema i omogućava jednostavnije proširenje u budućnosti.

6. Zaključak

Dodavanjem **Facade** i **Adapter** paterna StudyBuddy sistem postaje pregledniji, fleksibilniji i lakši za održavanje. Facade patern pojednostavljuje rad sa složenim procesima koji uključuju sesije učenja, korisnike, prijave i obavještenja. Adapter patern omogućava povezivanje sistema sa eksternim email servisom bez mijenjanja postojećih klasa.

Ovi paterni ne mijenjaju osnovni model sistema, nego ga nadograđuju dodatnim klasama koje poboljšavaju strukturu i pripremaju sistem za buduća proširenja. Zbog toga su **Facade** i **Adapter** prikladni strukturalni paterni za StudyBuddy sistem.